Sri Lanka Institute of Information Technology



Research Project (Comprehensive Design and Analysis Project) IT4010
Deployment Report

R26-SE-002

FraudAware – CareerNet

Team member	Student ID	Individual component
Kajanthan U	IT22224002	Scam Communication Detection
Santhosh S	IT22113740	Multilingual Fake Job Post Detection
Bandara H M N T	IT22251664	Employer Legitimacy Verification
Methmi N P C S	IT22188618	Risk-Aware Job Recommendation and Ranking

BSc (Hons) in Information Technology

Specialized in Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

Contents

1. Introduction and scope 2
2. Deployment architecture 2
3. Backend services on Railway 3
4. Message broker and service communication 3
5. Web frontend on Vercel 4
6. Mobile app on Google Play Console 4
7. Deployment methodology 5
8. Verification of the deployment 6
9. Challenges and limitations 7
10. Conclusion 7
Appendix A. Screenshots 8

1. Introduction and scope

The supporting backend services, the web frontend and the mobile app of FraudAware (group R26-SE-002) are deployed and publicly reachable. This report records where each part runs, how it was deployed, and how the deployment was checked.

The deployment uses four platforms:

- **Railway** hosts the backend services (user, notification, job management, chat, email) and the four main services: Fake Job Detection, Employer Legitimacy Verification, Scam Communication Detection and Risk-Aware Job Recommendation and Ranking.
- **Hugging Face** hosts the machine learning models used by the research components.
- **CloudAMQP** hosts the RabbitMQ message broker.
- **Vercel** hosts the web frontend.
- **Google Play Console** distributes the mobile app.

The remaining main research component services will be deployed later and are not part of this report. Component 1 is listed in section 3 because it is already deployed.

2. Deployment architecture

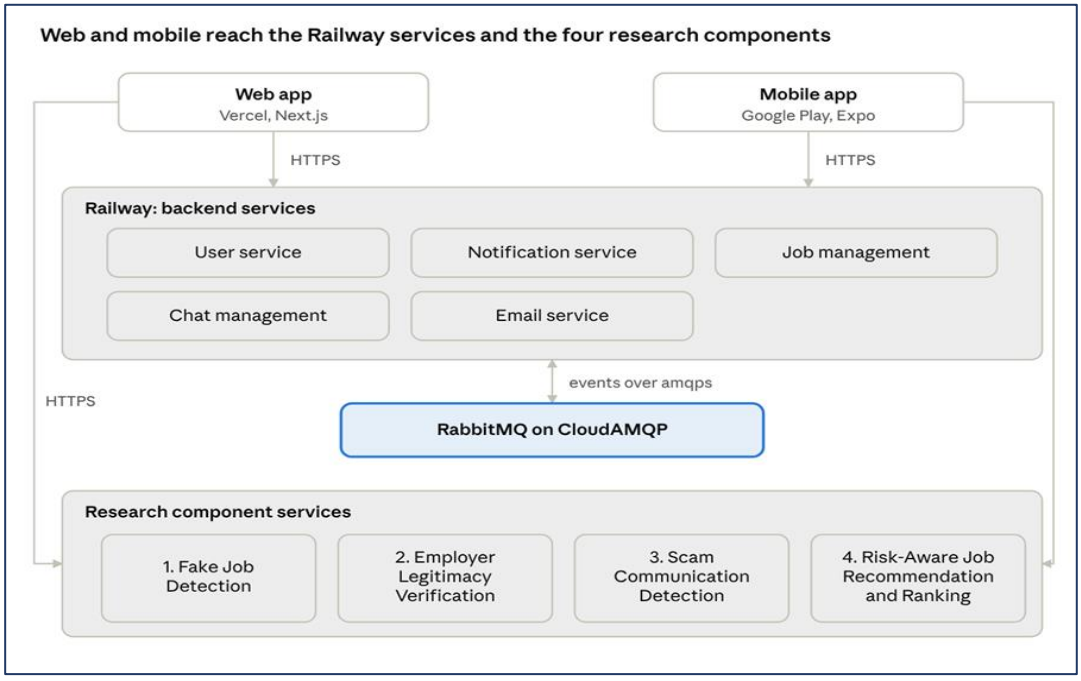


Figure 1: Deployment architecture · 4 platforms

The web and mobile apps call the Railway services over HTTPS, and the services exchange events through one RabbitMQ broker on CloudAMQP.

3. Backend services on Railway

Six services run on Railway, each with its own public *.up.railway.app domain.

Service	Role	Production URL
User service	User accounts and authentication	https://r26-se-002-production-97e2.up.railway.app/
Notification service	Notifications	https://r26-se-002-production-d56b.up.railway.app/
Job management service	Job postings	https://r26-se-002-production-a302.up.railway.app/
Chat management service	In-chat service between recruiter and candidates	https://r26-se-002-production-7780.up.railway.app/
Email service	Email sending	https://r26-se-002-production-22ab.up.railway.app/
Fake Job Post Detection (Component 1)	Fake job post detection	https://r26-se-002-production-dbd4.up.railway.app/
Employer Legitimacy Verification (Component 2)	Employer (company/business) legitimacy verification	https://r26-se-002-production-3ef6.up.railway.app/
Scam Communication Detection (Component 3)	Communication messages detection	https://r26-se-002-production-0b09.up.railway.app/
Risk-Aware Job Recommendation and Ranking (Component 4)	Aware risk and recommend jobs	https://r26-se-002-production-8270.up.railway.app/

4. Message broker and service communication

RabbitMQ runs as a managed instance on CloudAMQP (host seal.lmq.cloudamqp.com) and is reached over amqps, the TLS-encrypted AMQP protocol. Services exchange events through it and call each other's REST endpoints over HTTPS.

The broker connection string contains a username and password. It is stored only as the RABBITMQ_URL environment variable on each service and is deliberately not reproduced in this report.

5. Web frontend on Vercel

The Next.js web application (admin dashboard) is deployed on Vercel and calls the Railway services listed in section 3. The deployment process was:

1. Push the web application code to the Git repository.
2. Import the repository as a new project in Vercel.
3. Vercel detects the Next.js framework and builds the application.
4. Vercel deploys the build and gives it a public production URL.
5. Every later push to the connected branch is built and deployed again automatically.

Deployment details:

- Production URL: https://careernet-two.vercel.app/login (Admin Dashboard)
- Source repository and branch connected to Vercel: https://github.com/Kaju2002/R26-SE-002.git, Main branch

6. Mobile app on Google Play Console

The Expo (React Native) mobile app is built as an Android release and published through Google Play Console. In production, it calls the deployed backend services. The deployment process was:

1. Prepare the app for release: set the app name, package name, version and icons.
2. Build the Android release bundle (AAB) with the Expo build service.
3. Create the app in Google Play Console and complete the store listing (description, screenshots, privacy and content details).
4. Upload the release bundle to a release track in Google Play Console.
5. Submit the release for Google review.
6. After approval, the app is published and can be installed from the Google Play Store.

7. Deployment methodology

Every part followed the same pattern: build from source, inject configuration through environment variables, deploy to a managed platform, then verify.

1. **Prepare each service.** Each backend service reads its settings (database address, broker URL, secrets) from environment variables, so the same code runs locally and in production.
2. **Deploy backends to Railway.** Each service is deployed as its own Railway service and gets a public *.up.railway.app domain. Secrets are set in Railway's variables panel and never committed to the repository.
3. **Connect the message broker.** A managed RabbitMQ instance on CloudAMQP replaces a local broker. Its amqps URL is set as RABBITMQ_URL on the services that need it.
4. **Deploy the web app to Vercel.** The repository is connected to Vercel, and the Railway service URLs are set as environment variables.
5. **Build and publish the mobile app.** The Android release build is produced with the Expo build service, with production API URLs set at build time, then uploaded to Google Play Console.
6. **Verify.** Each public URL and each user flow is checked as described in section 8.

Open question: add the exact build commands and the start command used on Railway, if the report should list them.

8. Verification of the deployment

Fill in the result column after running each check against the live deployment.

Table 1: Verification of the deployment

Check	Method	Result
Each Railway service responds	Open its public URL and a health or root endpoint	Passed: all six Railway services responded on their public URLs.
Component 1 detection works	Submit a job post to the Component 1 service and read the prediction	Passed: the service returned a fake or real prediction for the submitted job post.
Component 2 employer verification works	Submit a company name to the Component 2 service and read the result	Passed: the service returned a verification result for the submitted company.
Component 3 scam communication detection works	Submit a message to the Component 3 service and read the result	Passed: the service returned a scam or safe result for the submitted message.
Component 4 job recommendation and ranking works	Request recommendations from the Component 4 service	Passed: the service returned a ranked list of jobs.
User registration and login	Register and sign in from the web app and the mobile app	Passed: registration and login worked on both the web app and the mobile app.
Messaging between services	Trigger an action that publishes a RabbitMQ message and confirm the consumer receives it	Passed: the message was published to RabbitMQ and received by the consuming service.
Web app reaches the backends	Use the Vercel site end to end	Passed: the Vercel site worked end to end against the Railway services.
Mobile app reaches the backends	Install the Play Store build and repeat the main flows	Passed: the Play Store build installed and the main flows worked against the backends.

9. Challenges and limitations

- **Missing secrets break features.** The user service failed on registration (`Missing credentials for "PLAIN"`) until mail credentials were provided, so every production variable has to be set before launch.
- **Expo version mismatch.** The project used Expo SDK 54 while the phone's Expo Go app was SDK 57, which blocked testing through Expo Go and led to building the app directly.
- **Slow employer check.** The employer verification check queries live registries and takes 20 to 50 seconds, so clients need a request timeout of 150 seconds or more and a loading state.
- **Free-tier hosting.** [Add any Railway, Vercel or CloudAMQP plan limits you hit, such as sleeping services or connection limits.]

10. Conclusion

The backend services run on Railway, messaging runs on CloudAMQP, the web app runs on Vercel and the mobile app is on Google Play Console, all configured through environment variables with no secrets in the code. Remaining work: fill in the verification results in section 8 and add the missing links.

Appendix A. Screenshots

(Add deployment screenshots here: Railway dashboard, Vercel dashboard, Google Play Console.)

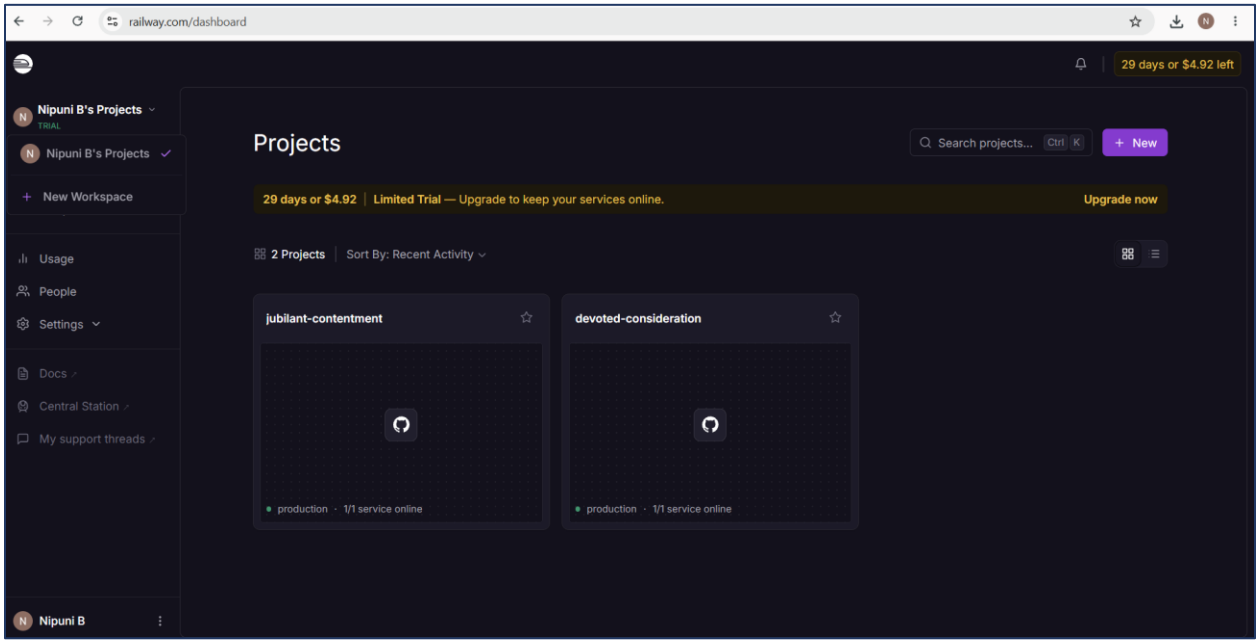


Figure 2: Railway Dashboard

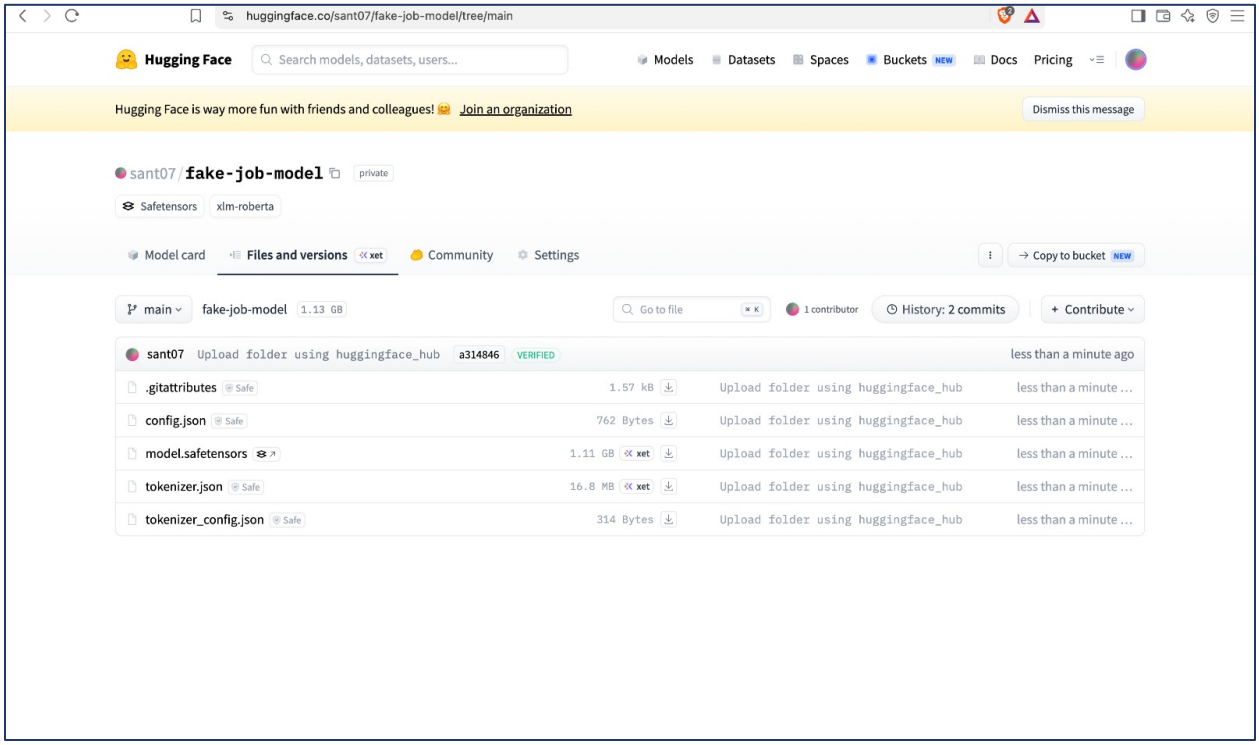


Figure 3: HuggingFace Dashboard

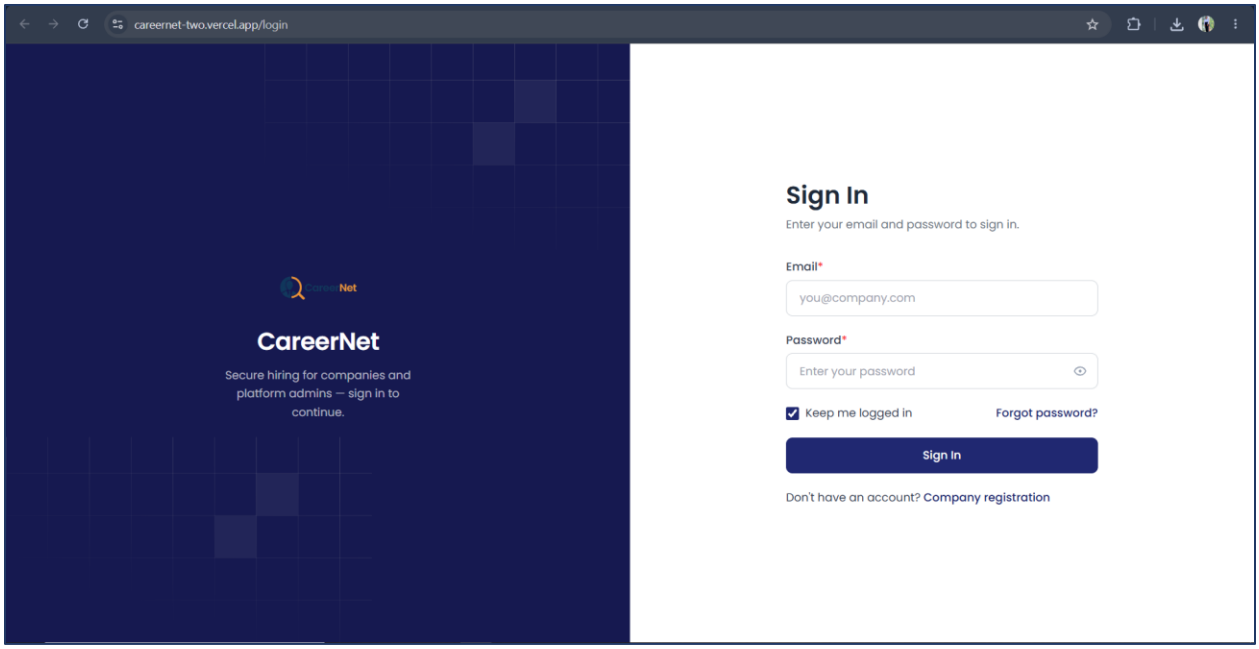


Figure 4: Vercel app running